

Project 2: Preparing Stock Data for Machine Learning

Hiren Patel
Technical Report

Notebook Link -

http://localhost:8888/notebooks/Stock_Market_Data_Part_2.ipynb

My overall approach

The goal of Project 2 was to take the stock-market data I explored in Project 1 and prepare it for machine learning. Instead of just describing the data, this time I cleaned it, created useful new features from the prices, combined the two files, and saved a final dataset that can be used to build models in Project 3. I did not build any model here — the focus was entirely on preparing high-quality data. I worked step by step so that every decision is clear and can be followed by someone with no technical background.

One important note about my data: in Project 2 I loaded the complete dataset of about 21 million rows, which is much larger than the smaller sample I had used in Project 1. Because of this, some of my numbers are different from Project 1 — this reflects the fuller dataset, not a change in method.

1. Data-cleaning decisions

I began by loading both files (the daily prices and the company information) and converting the date column to a proper datetime type so it could be used for time-based work. I then applied the cleaning decision I had identified in Project 1: I removed one accidental "Symbol" record from the company file, which was a stray header row mistakenly stored as data. I also checked that this junk value did not appear in the price file, and it did not, so the price data was already clean of it.

Finally, I sorted the price data by ticker and then by date. This step is essential, because the features I create next (like previous-day price and moving averages) depend on each stock's rows being in the correct time order.

2. Unusual observations investigated

In Project 1 I had found some very high prices. In this project I reviewed them again and decided to retain them, because I had already confirmed they are real stock prices, not

errors. The same spike appeared in both the close and adjusted close for the same stock, which is how genuine price movements behave.

Rather than delete these real values, I chose to flag them. I created two new columns that mark the most extreme prices (the top 0.1% of values). This way I keep 100% of the real data, but a future model can still see which rows are extreme and decide how to handle them. I was careful not to delete real data just to make the statistics look smaller, because that would bias the dataset and hide genuine market behaviour.

3. Features created

I created several new features from the price data to describe recent price movement and stock behaviour. These included the daily price change (in dollars), the daily return (as a percentage), a monthly return (about one month), the previous day's closing price, and two moving averages: a 5-day (about one week) and a 20-day (about one month). I also created a volatility measure, which is the standard deviation of daily returns over 20 days, showing how much a stock bounces around.

For my additional features I focused on ones that work fairly across cheap and expensive stocks. I created a percentage high-low range, which shows how much a stock swings during a single day as a percentage. I also created volume features: the change in trading volume from the previous day, the 20-day average volume, and a relative volume measure (today's volume compared to the stock's own 20-day average). I chose relative volume because raw volume cannot be compared between stocks — a cheap stock naturally trades far more shares than an expensive one — so comparing each stock to its own normal level is a fairer, stronger signal. I calculated every feature per stock, so one company's values never mix with another's.

4. Combining and organizing the data

Next I combined the useful company information (exchange, sector, and industry) into the price data, using the ticker as the key. Instead of a normal merge, I used a lookup-and-map approach. I did this for a very practical reason: a normal merge builds a second copy of the 21-million-row table in memory, which repeatedly ran out of memory and crashed my environment. The map approach adds the columns directly without copying the whole table, so it uses far less memory, and it also cannot accidentally duplicate rows. I confirmed the join was clean by checking that the row count stayed exactly the same after combining.

During this stage I also moved my work from a cloud notebook (Google Colab, which has limited memory) to a local Jupyter notebook on my own computer, which has much more memory. This solved the crashing problem and let me process the full dataset smoothly.

5. Handling missing values

The new features I created produced missing values at the very start of each stock's history, because features like moving averages and lags need earlier days that do not exist yet. I considered three options. First, filling them with zero — I rejected this because a zero return or zero average never actually happened and would mislead the model. Second, using a growing average that uses however many days are available — a valid option, but the earliest values would be based on very few days and less reliable. Third, dropping the warm-up rows — which is what I chose, because it keeps my data the cleanest, with every remaining row having fully valid features.

This removed 113,555 rows out of about 21 million — less than 1% of the data — which is a small, safe loss in exchange for a clean, reliable dataset. I did not replace any values with zero, as the assignment specifically warned against.

6. Encoding categorical variables

Machine-learning models work with numbers, not text, so I converted my categorical columns (exchange and sector) into numbers using one-hot encoding. First I filled the 1,440 missing sector values with an "UNKNOWN" label, so those stocks became their own group instead of being lost. One-hot encoding then created a separate 0/1 column for each category.

I chose one-hot encoding instead of simply numbering the categories (like NASDAQ = 1, NYSE = 2) because plain numbers would accidentally create a false ranking — a model would think NYSE is "greater than" or "twice" NASDAQ, which is meaningless. One-hot encoding gives each category its own separate column, so no category is treated as bigger or smaller than another. I did not encode the industry column, because it has 136 different values and would add far too many columns.

7. Final dataset

For the final dataset I dropped the columns a model cannot use — the company name (just an identifier) and the industry column (too many unencoded categories). I kept the identifiers I need (ticker and date), the price and volume columns, all my engineered features, the extreme-value flags, and the one-hot encoded exchange and sector columns.

Before saving, I checked the final dataset. It has 20,860,334 rows and 36 columns, with the correct data types, no remaining missing values, no duplicate ticker-and-date combinations, and dates spanning 1970 to 2018. I then saved it as `stock_prepared.csv`, which will be the starting point for building machine-learning models in Project 3.

8. Key findings

By following this step-by-step process, I turned the raw stock data into a clean, fully numeric, model-ready dataset. The data is real and mostly clean, the extreme prices were kept and flagged rather than deleted, and all the new features were calculated per stock and normalized where possible so they work fairly across cheap and expensive stocks. The biggest practical lesson was about memory: working with 21 million rows required careful choices — using a map instead of a merge, dropping warm-up rows instead of filling them, and moving to a machine with enough memory — to prepare the data without losing quality. The result is a dataset that is ready for the modelling work in Project 3.